Professional AI Training
(Covers Prompt Engineering, Context Engineering, Spec and Intent Engineering along with Agentic AI capabilities - RAG, MCP and Workflows)
Facilitator: Ramkumar J Dhamodharan

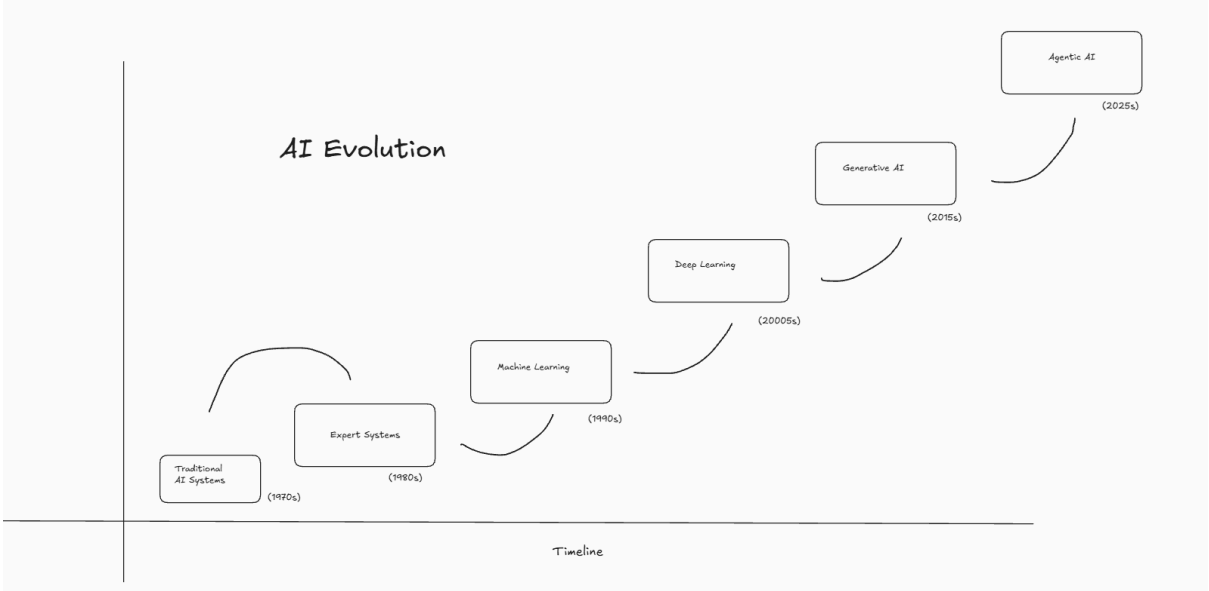
Training Agenda - Day 1

- AI Evolution
- LLM Foundations
- Engineering Disciplines with AI
- Agentic AI
- Orchestration Frameworks (LangChain / LangGraph)
- Setup Development Environment
- Practice #1

By end of the day, you shall be able to do five things

- Explain how AI evolved and why Agentic AI is the current shift
- You shall understand what's happening inside a LLM when you send a request, what each of the common settings actually controls
- Engineering Disciplines - Prompt, Context, Spec and Intent -> Harness Engineering
- You shall be able to tell an agent from a workflow - how they're different
    - When to choose one?
- Prompt | Context Engineering - Techniques
    - Demonstrations

AI Evolution - Why the story of AI Matters?



ERA 1:
Traditional AI Systems

- Rule based AI
- No Learning | No Training on Data
- Humans think, humans write the rule and the machines follow them
    - Perfectly every single turn

Expert Systems

- Software built to capture the knowledge of a specialist in one narrow field
- 3 Elements
    - Knowledge Base
    - Inference Engine
    - User Interface

+

- Transparent

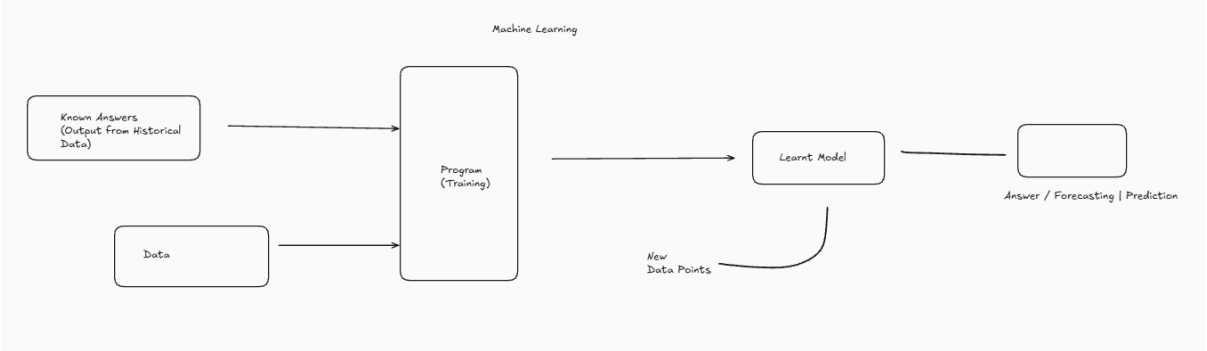
- Predictable
- Fast & Cheap to run

-
- Brittleness
- Knowledge Bottleneck - Time Consuming - Difficult to document
- Maintenance

ERA 2:

Machine Learning - Learning Rules from Data

- Data Driven Learning
- Understand Intricate relationships within the data - Patterns within the data - how data points are correlated



- Learning by Example - Not by Instructions

Three Different styles of Learning

- Supervised Learning
- Unsupervised Learning
- Reinforcement Learning (Semi-Supervised) - RLHF

-
- Continuous Learning
- Manual Feature Engineering
- Scalability
    - CPU based Learning (At initial time)
    - Time-to-Market
    - Cost
    - Training Time / Cost
    - Productivity
- Well suited for Structured Data (such as Cleaned Dataset | DB Table)
    - May NOT work at performant - on Unstructured | Semi-structured Data

ERA 3:
Deep Learning

- Subset of Machine Learning
- Data Driven Learning
- It's ML using Neural Networks with many layers stacked up on the top of each other -
    - Input Layer
    - Hidden Layers (Deep)
    - Output Layer
- Hierarchical Relationships within the data
- Computer Vision
- Speech
- Tremendous Improvements on Unstructured Data
    - Text
    - Image
    - Video
    - Speech
    - Code

+

- Automatic Feature Engineering
- GPU Hardware Acceleration
    - Training Time
    - Time-to-Market
    - Training Cost

-

- Training Cost (Using GPU Hardware is Expensive)
- Continuous Learning

Era 4:
Generative AI

- The Transformer and Attention
- Attention is All you Need - Transformer
- Attention - It's the mechanism that lets the model make the same connection, by weighing the relationship between a single word to other words in a passage and every other word around it.
- From Predicting to Generating
- Language Model
    - A Deep Learning Model which has abilities to NL*
        - NLU (Understanding)
        - NLP (Processing)
        - NLR (Response)
    - Has abilities to understand, process and generate new response content how humans behave
    - Generating New Contents from Existing Trained Data
- Transformer Explained Visually
    - https://poloclub.github.io/transformer-explainer/

Language Model

- Foundational Model
- One Large Language Model, pre-trained on broader | global | humongous amount of data, that can be adapted to thousands of different tasks with just an instruction
- A Single foundation model performs various kinds of tasks
- GPT (Generative Pre-Trained Transformer) - November 2022 - GPT 3.5

Era 5:
Agentic AI

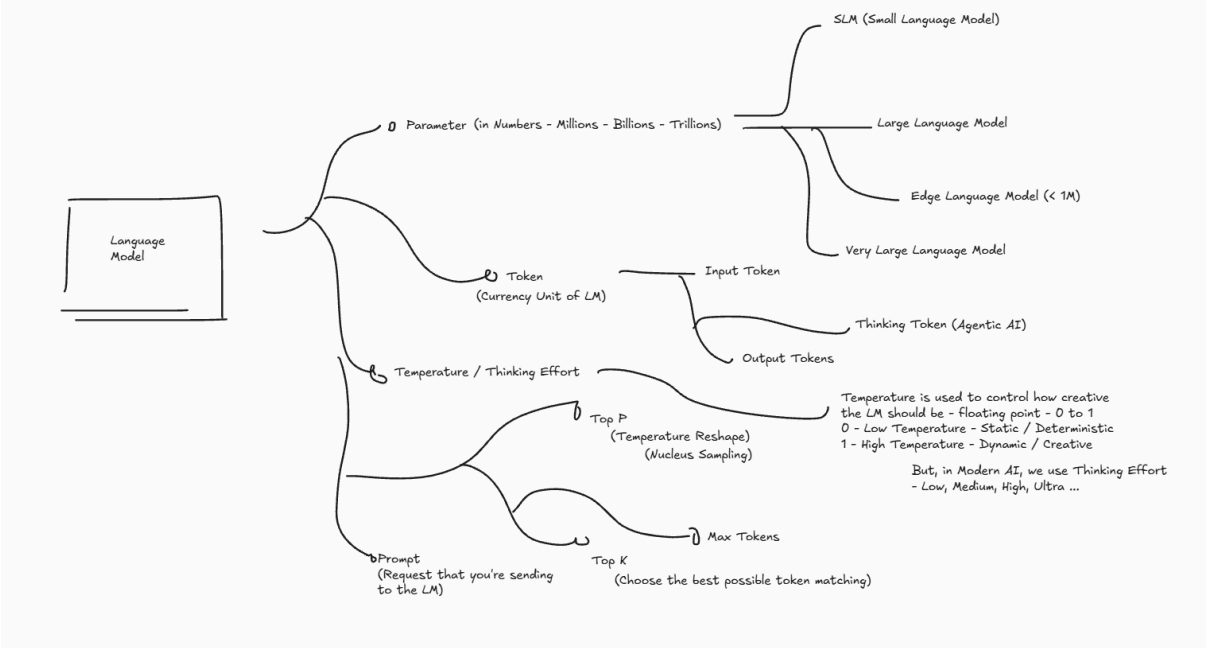
From Answering to Actions

- It's about moving from a model that answers to a system that acts
- An agent is given a goal, not just a question
- It works out the steps required to accomplish the goal with the help of Reasoning Models - uses tools to take actions - looks at the results - and adjusts its plan until the job is DONE.
- **ReAct Agents**
    - **Re- Reasoning**
    - **Act- Actions**
- Tools, Memory, Persistence, State Management and HITL

LLM Fundamentals

Language Model

- DL Model with NL* capabilities



Common Terms in LMs

- Based on
  - Parameter Size
    - SLM, VLLM, ELM, LLM
  - Generation Type
    - Singularity Model
      - Take One type of Input / output
    - Multi-Modality Model
      - Take any kinds of Inputs / Outputs
- MoE (Mixture of Experts)
- Model Inference
  - Mechanism of Hosting a Model for Client Access
- Open Source | Closed Source Models
  - Open Weight Models (Gemma, Llama Models)
  - Closed Models (GPT, Claude, Gemini)
- Reasoning Models
  - Language Models that have the ability to break a problem statement into multiple easy to execute sub tasks -> arrange them and parallely answer them
  - CoT (Chain of Thoughts)
  - ToT (Tree of Thoughts)
- Tool Calling Features in LM

- A program that has all the tools information (Only Metadata) attached to a prompt, and letting the LM do reasoning / planning,so the program can execute those tools to get more information about the prompt (in the form of context), so answers can be generated dynamically without always referring to global knowledge.

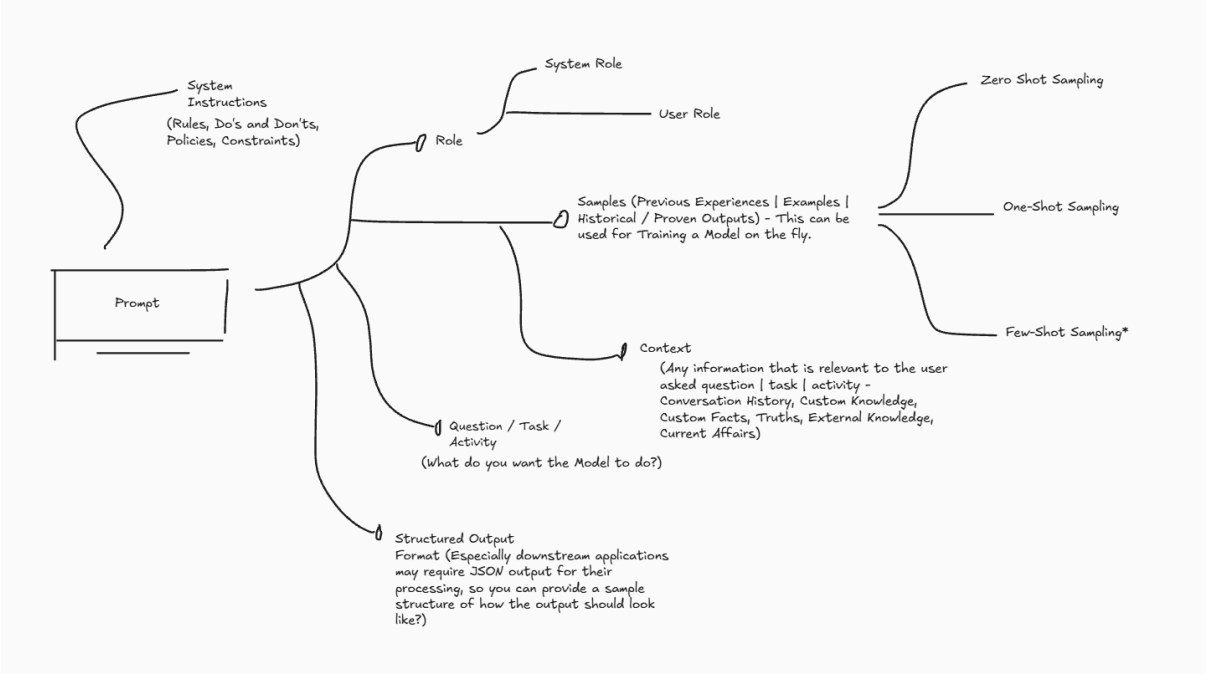
LMs

- Read Only
- Time Frozen (Time-Cut-off)
- Stateless

Hallucinations & Knowledge Cut-Off

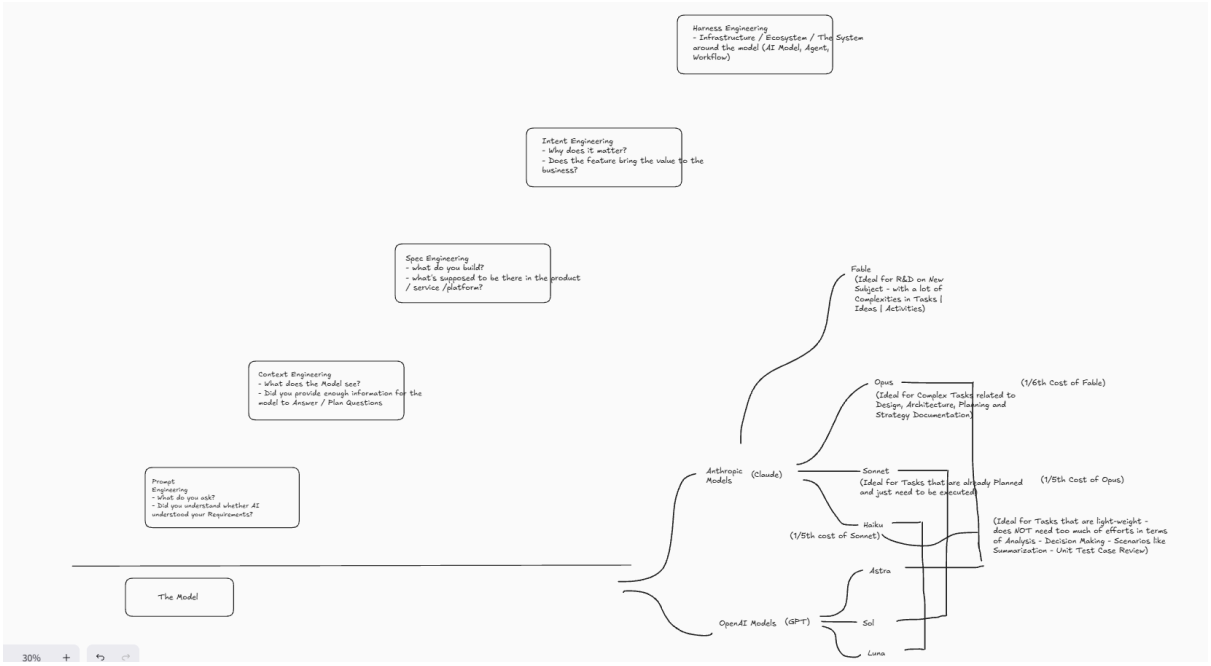
- It’s when a model produces something fluent, confident and false
- Prompt Control
- Context Information
- System Instructions
- Judge LLM for Evaluation

Prompt - Components



AI Engineering Disciplines

- Prompt Engineering, Context Engineering, Intent and Spec Engineering -> Harness Engineering
- The Five Layer Ladder



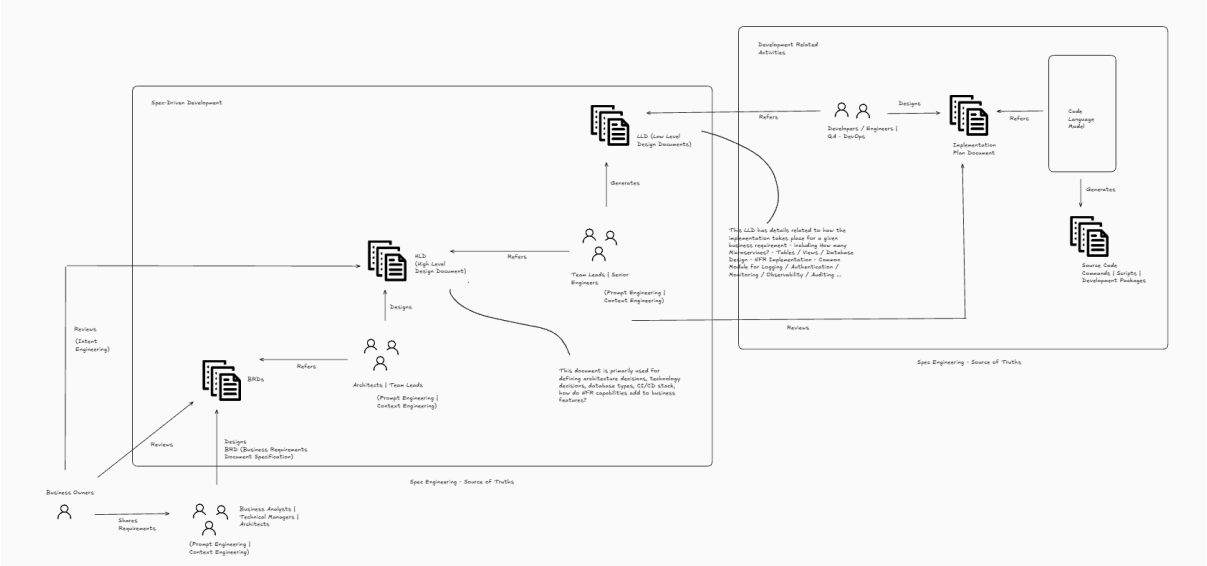
Context Engineering

- It’s the practice of designing and assembling everything the model sees at the model when it generates the response, NOT just the instruction you wrote.
- A typical context may include the following (NOT ALL OF THEM)
    - System Instructions
    - Conversation History
    - Retrieved Documents (for RAG)
    - Tools Metadata & Tool Execution Results
    - Memory
        - Episodic Memory
        - Short-Term Memory
        - Long-Term Memory
    - Examples

If Context is SCARCE, how do you manage them?

- In general, you apply 4 different strategies
    - Select
        - Vector Search | Similarity / Semantic Search
        - Term Search | Hybrid Search
    - Compress
        - Shrink what you keep - Instead of carrying a lengthy conversation history - summarize and keep them
    - Order
        - Position Matters
        - Put the most important instructions and facts where the model is MOST likely to use them - START / END
    - Isolate
        - Split work so that each part gets a clean - focused context

Spec Driven Development



Prompt and Context Engineering - Best Practices - Do’s and Don’ts

- Philosophical Thoughts
    - Positive Words in a Prompt - GOALS
    - Negative Words in a Prompt - CONSTRAINTS

- In Prompt Engineering, Always follow 6 IMPORTANT Good Habits
    - Lead with the Goal, and the Audience
    - Specify the format and the Length
        - For example, Markdown File Format & within 50 Lines
    - Tell the Model what to do, when it does NOT know
    - Separate Instructions from the Data
    - Say what to do, NOT only what to avoid
    - Manage Prompts like Code
        - **Prompt Library with Versioning Control**

- Not so GOOD Habits
    - Do NOT expect the model to read your mind
    - Do NOT give contradictory instructions
    - Do NOT shout
        - Use Calm, clear language and if a rule really matters, explain why it matters.
    - Trust that AI can do any work, but do NOT trust the output blindly
        - HITL
        - Judge LLM
        - Evaluation
        - Lookup Citations with Sources

Context Engineering - Do's and Don'ts

Five Good Habits:

- Curate, Don't collect
    - Less, but more relevant, always almost wins
- Put the things where the model will use them first
- Label your sources
- Ground the model, and give it permissions to say "I don't Know"
- Manage memory deliberately

Four Not So Good Habits:

- Don't dump everything just because the context window is big
- Do NOT feed stale or conflicting documents
- Do NOT mix trusted instructions with UNTRUSTED content
    - Prompt Injection
    - Input Guardrails
    - Evaluations
    - Caching
- Do NOT ever let a long conversation ROT

In the cloud lab,

-> Task Bar -> Open Terminal Window

> cd c:\users\admin\lab-systems

> pwsh --version
(two dashes)

> start -> search for Docker Desktop

> make sure that it's running

> pwsh .\installation-check.ps1

You should now be able to see all software items are INSTALLED and CONFIGURED. You can ignore SSMS related failure messages, as they're NOT relevant to this training.

> Open VS Code

> View -> Chat

> select Models -> Configure Models

> In the new Models Dialog, Add Models

> Select Anthropic

> It shall be prompting in the top navigation bar for Title

> Press <ENTER>

> for API Key, paste the following

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

> Press <ENTER>

> Once the models are loaded, select Sonnet 5.5 in the chat window

> paste the following prompt

can you describe about Prompt Engineering Good Habits in 5 bulleted points? I need brief answer, and NOT lengthy responses.

Anthropic Claude AI Tools & Services - Roles Specific

Claude Tool / Services	Roles & Purpose
Claude Desktop	Business Owners / Business Analysts | Technical Managers | Architects - Non Technical Use (Document Summarization, Designing Presentation Decks, Designing BRDs, HLDs and LLDs)
Claude Code	Developers | Technical Engineers - Who wish to perform technical activities such as Programming, UI Development, Testing - Packaging and Deployment
Claude Code CLI	Client Command Line Utility Edition for Claude Code
Claude Code for VS Code	VS Code Integrated Claude Code
Claude Design	UI/UX Designer - Engineer
Claude Projects	Artifacts Management Workspace for Multiple Claude Sessions
Claude Cowork	AI assistant to perform common tasks in your system (such as Browser activities, Email activities)
Claude Dispatch	An integrated tool using which Claude Mobile | Claude Web can be easily used to communicate with Claude Code Sessions | Computer

OpenAI Tools & Services - Roles Specific

OpenAI Tool / Services	Roles and Purpose
OpenAI ChatGPT Desktop	Business Owners / Business Analysts | Technical Managers | Architects - Non Technical Use (Document Summarization, Designing Presentation Decks, Designing BRDs, HLDs and LLDs)
OpenAI Codex	Developers | Technical Engineers - Who wish to perform technical activities such as Programming, UI Development, Testing - Packaging and Deployment
OpenAI Code CLI	Client Command Line Utility Edition for Claude Code
GitHub Copilot (Microsoft & GitHub)	Developers | Technical Engineers - Who wish to perform technical activities such as Programming, UI Development, Testing - Packaging and Deployment
GitHub Copilot CLI	Client Command Line Utility Edition for Claude Code
OpenAI Codex and GitHub Copilot for VS Code	Integrated OpenAI Code and Copilot with Visual Studio Code

Day 2 Agenda

- Day 1 MCQs with Participants
- Spec Driven Development - Demonstration Continued
    - HLD
    - LLD
    - Implementation Plan
    - Development | Deployment
        - Back-End & Front-End
- Lecture on Context Engineering Deep Dive - RAG
- Lecture on LangChain | LangGraph
    - Introduction
    - Features
    - What's there in the Box?
    - When to use What?
- Demonstration on RAG with LangChain & Agents

```
stateDiagram-v2
    direction LR
    [*] --> Active : Customer creates goal
    Active --> Achieved : Balance reaches target
    Achieved --> Active : Withdrawal below target / Target raised
    Active --> Cancelled : Customer cancels (balance ₹0)
    Cancelled --> [*] : Customer deletes goal

    note right of Active
        May show Off-track or Overdue indicator
    end note
```

FMR AI Use Case 1 - Spec Driven Development
https://github.com/iomegak12/fmr-ai-usecase-1

Points to Ponder in Prompting Techniques

- Do NOT let AI models assume anything.
    - If anything requires to be clarified, tell AI models to clarify with you.
- Do NOT hurry / rush in Response | Output Generation
    - Always clarify what it's going to generate, review them before it starts doing the work of Generation.
- Do NOT always ask questions to AI, instead turn the AI models asking questions back to you.
    - Have an interview based | MCQs based QAs session, so both of you ARE clarified
- Enable AI models to prompt you MCQs based QAs, instead of prose style, which is convenient and comfortable
- Always prefer MERMAID diagram contents
- Always prefer MARKDOWN file format, instead of WORD | PDF | Any other document formats. (Later markdown files can be exported to PDFs)
- Whenever you have a pre-determined | existing technology stack | decision documents, feed them as CONTEXT to the prompt, so it does NOT have to question you again.
- Whenever there's a requirement to update / delete any sections of the contents generated earlier, ensure that prompt includes SECTIONS to reuse the contents (So you shall always have deterministic content, instead of generating everything new), and review the sections that are being updated in the document. After reviewing, enable the AI models update / delete the contents
- Until it's required, always enable AI models to brief as simple as possible, instead of generating paragraphic answers, which most of times bloated | NOT useful | may not be able to read everything.
- Instead of waiting for the complete output generated after you send a prompt, if there's a requirement to update the prompt, you can ALWAYS escape | cancel the execution, and resend the updated prompt, which shall take the effect of accumulating them to the earlier prompt.
- Trust the AI model that it can do ANYTHING (Input) related to IT, but NEVER trust the AI Model that it has DONE the WORK.
    - We don't want to review everything, but we should REVIEW what it really matters to us
- When you generate related documents (for example, LLD generated out of HLD, HLD document is generated out of BRD), it's a good practice to apply DRIFT checks with a different model to validate how accurate / aligned they're. It's BETTER to use a higher model (may be a different model provider)

Implementation Plan - Lessons

- As a Product Manager / Team Lead | Architect, I strongly recommend to design BRDs, HLDs, LLDs before development team crafts Implementation Plan Guide
- It's highly recommended to maintain repository of SKILLS which can be reused, shared and governed uniform across team members
- It's also a good practice to have repository with version control system - keeping track of all proven | well-known | common tasks specific prompts i.e. Prompt Library
- Add Knowledge Base to all - Prompt Libraries, Skills Repository and Code Repository - So any team member would be able to easily ask questions related to anything and avail those artifacts without having to communicate with other members
- Development Team - I would recommend them to refer 3 Documents (BRDs, HLDs, and LLDs) for their work related artifacts, and design Implementation Plan Guide with all required features (For example, Rate Limiting, Documentation, OpenAPI Endpoints, Swagger Support, Health and Ready Endpoints, Metric Endpoints, Logging, Authentication, Authorization, Auditing, Monitoring, Observability, Traces and so on)
- Team Leads | Architects are strongly recommended to review them before they're given to the Code Language Model for Code Generation | Application Development | Packaging | Deployment
- Development Teams when they have access to multiple AI Models (different service providers), It's strongly recommended to generate a varying Implementation Plans and analyze the drift. If the drift is MORE THAN 20%, definitely, we should start considering a specific model which has provided business specific documents | contents.
- It's strongly RECOMMENDED to use High-End models for Planning | Implementation Plans | Analysis | Research (For example, Claude Opus, OpenAI Astra …)

Lab Exercises on AI Assisted Development

> in the cloud lab, open the terminal window

> md c:\exercises

> cd c:\exercises

> git clone https://github.com/iomegak12/fmr-ai-usecase-1

> cd fmr-ai-usecase-1

> explorer .

> remove all the folders except docs (including .github)

> code .

> in the vs code, you should now be able to see docs folder, NOTHING ELSE

> in the chat window, ensure that you selected Sonnet Model

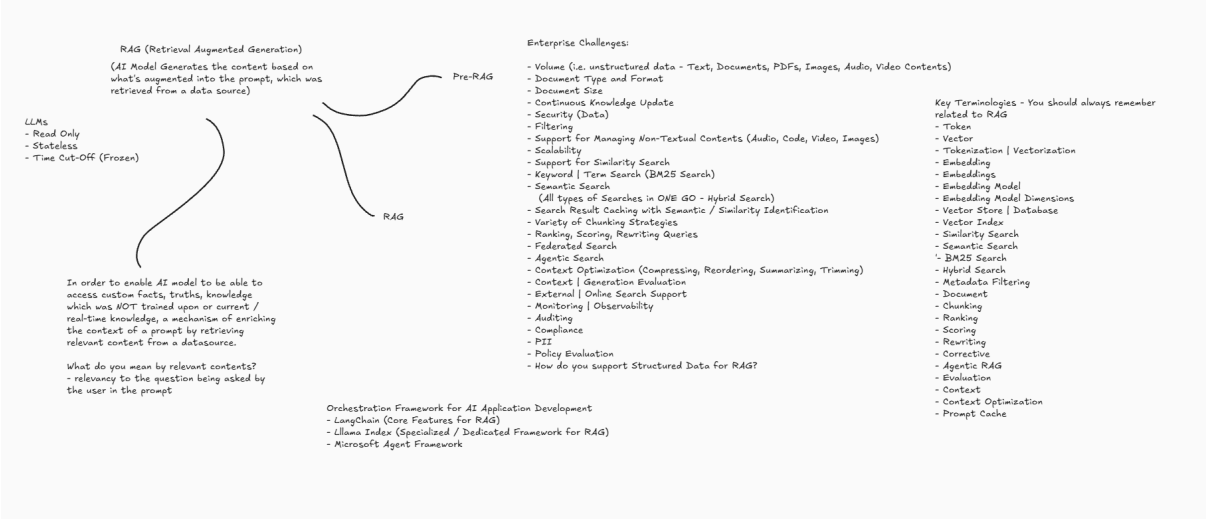
Exercise Requirement:

> add the following requirement to the specification

Authentication Services to both Goals and Transactions Services
JWT based authentication to both REST Services
User Profile Management (create profile, update profile and sign-in, reset-password)
No 2FA (Two Factor Authentication) required
Authentication service is an INDEPENDENT service, which is NOT specific this project requirements
By default user profiles are managed in Postgres Database

> Once the implementation has been updated with the above requirements, implement both Authentication and Goals Service by yourself using AI Assisted Development

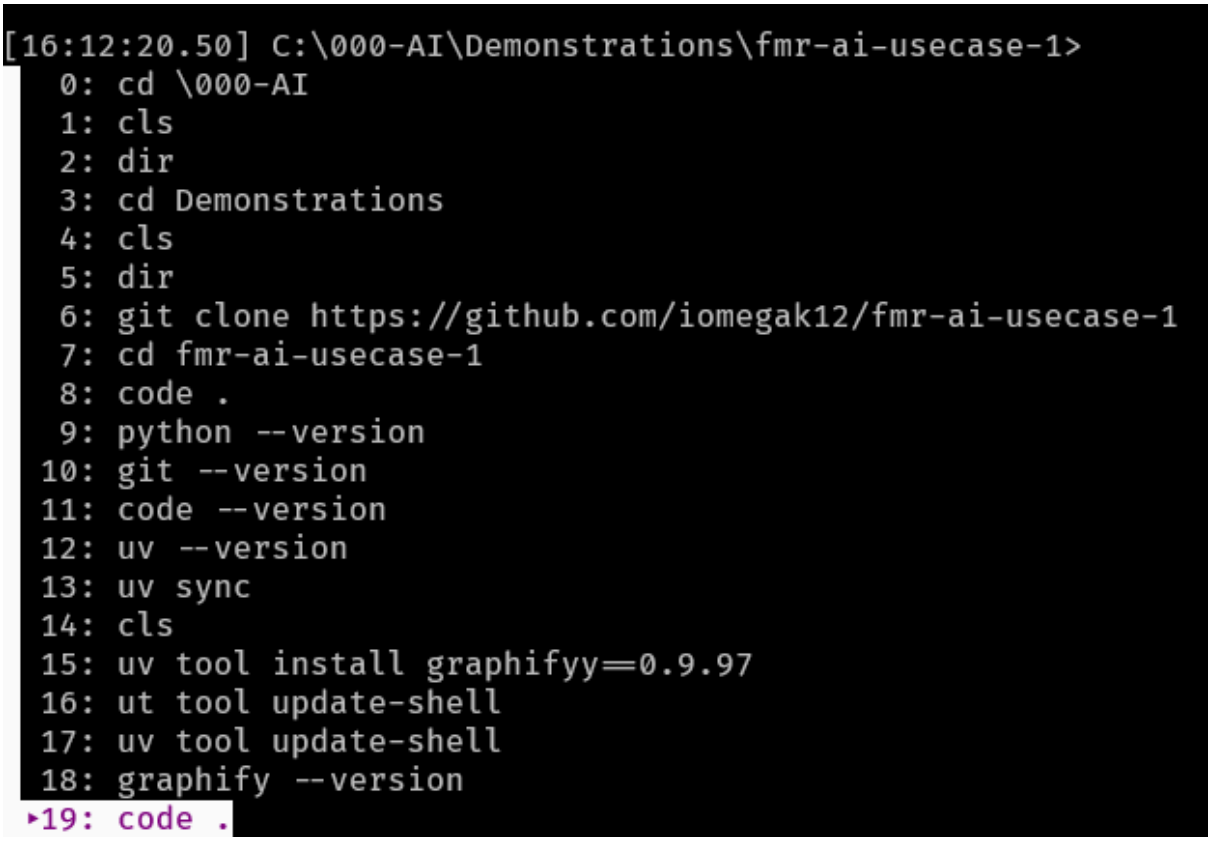
RAG


RAG (Retrieval Augmented Generation)
(AI Model Generates the content based on what's augmented into the prompt, which was retrieved from a data source)
Pre-RAG
RAG
LLMs
- Read Only
- Stateless
- Time Cut-Off (Frozen)
In order to enable AI model to be able to access custom facts, truths, knowledge which was NOT trained upon or current / real-time knowledge, a mechanism of enriching the context of a prompt by retrieving relevant content from a datasource.
What do you mean by relevant contents?
- relevancy to the question being asked by the user in the prompt
Enterprise Challenges:
- Volume (i.e. unstructured data - Text, Documents, PDFs, Images, Audio, Video Contents)
- Document Type and Format
- Document Size
- Continuous Knowledge Update
- Security (Data)
- Filtering
- Support for Managing Non-Textual Contents (Audio, Code, Video, Images)
- Scalability
- Support for Similarity Search
- Keyword | Term Search (BM25 Search)
- Semantic Search
(All types of Searches in ONE GO - Hybrid Search)
- Search Result Caching with Semantic / Similarity Identification
- Variety of Chunking Strategies
- Ranking, Scoring, Rewriting Queries
- Federated Search
- Agentic Search
- Context Optimization (Compressing, Reordering, Summarizing, Trimming)
- Context | Generation Evaluation
- External | Online Search Support
- Monitoring | Observability
- Auditing
- Compliance
- PII
- Policy Evaluation
- How do you support Structured Data for RAG?
Orchestration Framework for AI Application Development
- LangChain (Core Features for RAG)
- Llama Index (Specialized / Dedicated Framework for RAG)
- Microsoft Agent Framework
Key Terminologies - You should always remember related to RAG
- Token
- Vector
- Tokenization | Vectorization
- Embedding
- Embeddings
- Embedding Model
- Embedding Model Dimensions
- Vector Store | Database
- Vector Index
- Similarity Search
- Semantic Search
- BM25 Search
- Hybrid Search
- Metadata Filtering
- Document
- Chunking
- Ranking
- Scoring
- Rewriting
- Corrective
- Agentic RAG
- Evaluation
- Context
- Context Optimization
- Prompt Cache

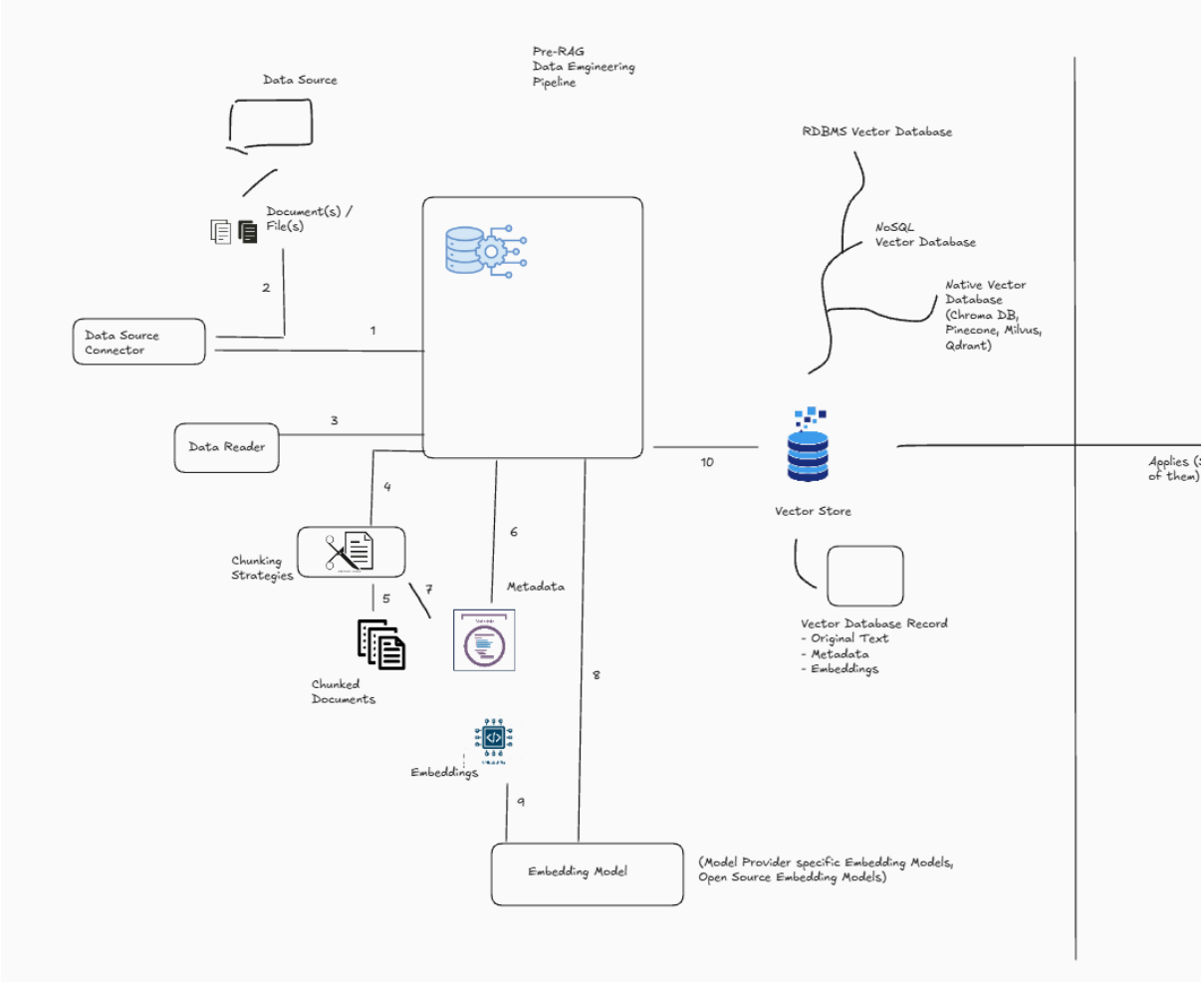
Day 3 Agenda

- Summarize - Day 1 and Day 2 Contents
- Lecture on RAG
- Demonstration & Practice on Agentic AI with LangChain & LangGraph
- Handling Brownfield Projects
    - Provisioning Knowledge Base for Existing Repositories
- Agentic AI Skills Development
- MCP Servers along with Agentic AI and
    - Monitoring | Observability | Evaluation Process

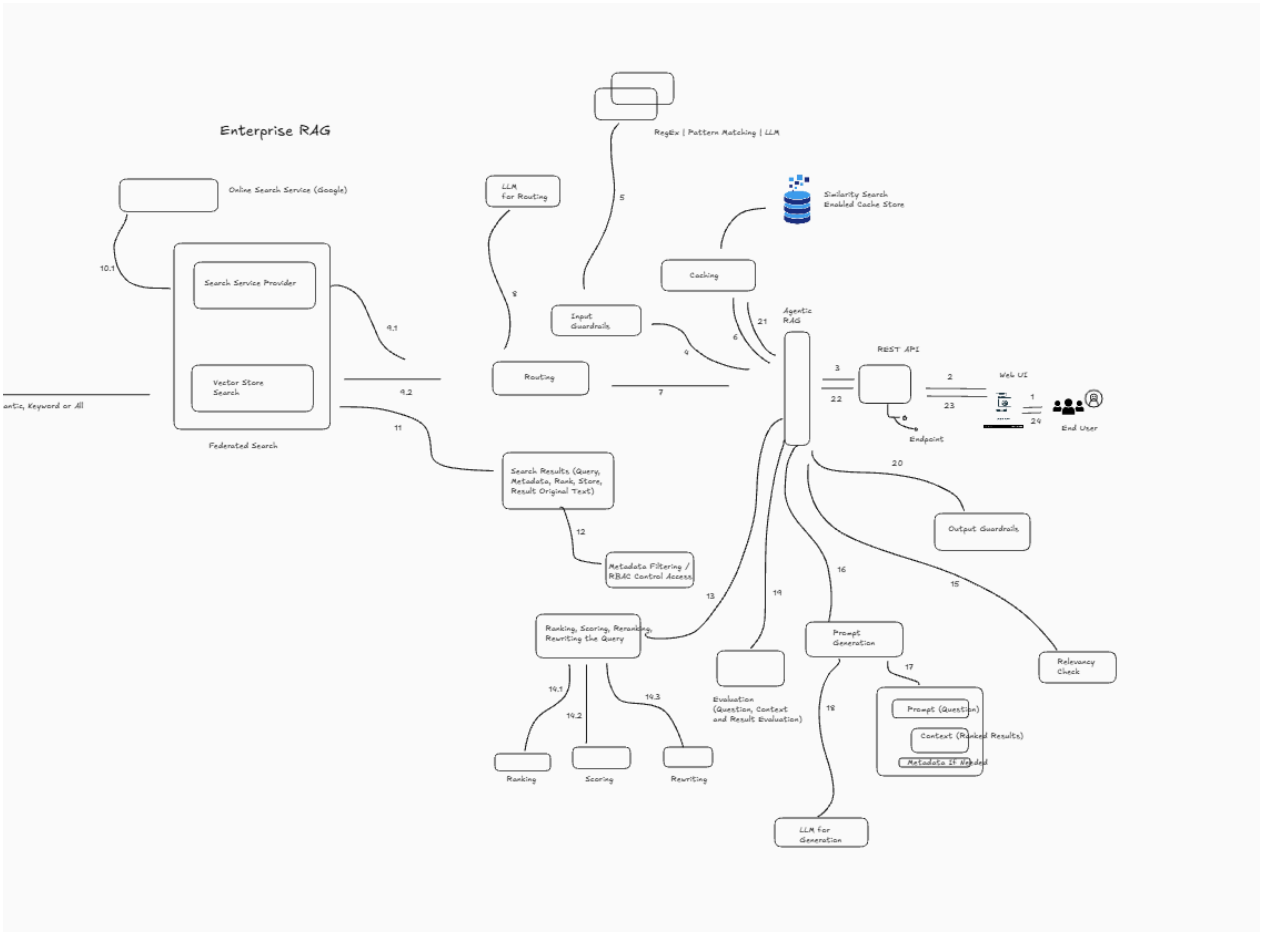
Graphify Tool Usage



RAG Architecture



https://excalidraw.com/#json=oY7O-OmmEOol0IZSwVmQi,rQO96UPtq2mkiBBtvlREAQ


Enterprise RAG
RegEx | Pattern Matching | LLM
Online Search Service (Google)
LLM for Routing
5
Similarity Search Enabled Cache Store
10.1
Search Service Provider
Caching
8
Input Guardrails
Agentic RAG
9.1
6
21
4
Routing
REST API
3
22
2
23
Web UI
1
24
End User
Endpoint
7
antic, Keyword or All
Vector Store Search
9.2
Federated Search
11
Search Results (Query, Metadata, Rank, Store, Result Original Text)
20
Output Guardrails
12
Metadata Filtering / RBAC Control Access
13
19
16
15
Ranking, Scoring, Reranking, Rewriting the Query
Prompt Generation
Relevancy Check
17
14.1
14.2
14.3
Evaluation (Question, Context and Result Evaluation)
18
Prompt (Question)
Context (Ranked Results)
Metadata if Needed
Ranking
Scoring
Rewriting
LLM for Generation

Agentic RAG - Practices

> In the cloud lab, Open the terminal

> md c:\demonstrations\agentic-rag

> cd c:\demonstrations\agentic-rag

> uv init --python=3.12

> uv sync

> code .

> in vs code -> left navigation -> extensions

> search for python

> and ensure that you activate the python extension at the workspace level

> search for jupyter

> and ensure that it’s also activated automatically

> view -> command palette

> search for python select interpreter

> choose python virtual environments -> select the virtual environment specific to the current workspace (agentic-rag)

> if you want to verify whether the virtual environment is activated, view -> terminal

You should now be able to see folder specific virtual environment name should be appearing the in the prefix

> in the vs code terminal, (view -> terminal)

> uv add pypdf reportlab

> create a folder named scripts

> create a file named generate_mf_docs.py

```
"""Generate three fictional mutual fund product PDFs into docs/ for RAG experiments."""

from pathlib import Path

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import cm
from reportlab.platypus import (
    KeepTogether,
    Paragraph,
    SimpleDocTemplate,
    Spacer,
    Table,
    TableStyle,
)

DOCS_DIR = Path(__file__).resolve().parent.parent / "docs"
DISCLAIMER = (
    "Fictional product document created for software testing only. "
    "Not investment advice. Figures are illustrative."
)
RISK_LEVELS = ["Low", "Low to Moderate", "Moderate", "Moderately High", "High", "Very
High"]

FUNDS = [
    {
        "file": "aurora_bluechip_equity_fund.pdf",
        "name": "Aurora Bluechip Equity Fund",
        "category": "Equity - Large Cap Fund",
        "tagline": "An open-ended equity scheme predominantly investing in large-cap stocks",
        "facts": [
            ("Launch date", "14 March 2012"),
            ("Benchmark", "Nifty 100 TRI"),
            ("Fund manager", "Rhea Kapoor (since Jan 2019)"),
            ("AUM (30 Sep 2026)", "INR 8,420 crore"),
            ("NAV - Direct Growth", "INR 142.37"),
            ("Plans / options", "Regular and Direct; Growth and IDCW"),
        ],
        "objective": (
            "The scheme seeks to generate long-term capital appreciation by investing at least "
            "80% of its assets in equity and equity-related instruments of the top 100 listed "
            "companies by market capitalisation. There is no assurance that the objective will "
            "be achieved."
        ),
        "strategy": [
```

```
        "Aurora follows a quality-at-reasonable-price approach. The investment team
screens "
        "for companies with consistent return on equity above 15%, low leverage and "
        "durable competitive advantages. Portfolio construction is bottom-up, with sector "
        "exposure capped at 25% of net assets.",
        "The portfolio typically holds 35 to 45 stocks. Turnover is kept low, with an "
        "average annual portfolio turnover ratio of about 0.28 times. The fund may use "
        "index futures for hedging up to 10% of net assets.",
    ],
    "allocation": [
        ("Large-cap equity (top 100)", "80% - 100%", "Moderately High to High"),
        ("Mid and small-cap equity", "0% - 20%", "High"),
        ("Debt and money market", "0% - 20%", "Low to Moderate"),
        ("REITs / InvITs", "0% - 10%", "Moderate"),
    ],
    "risk_level": 4,
    "risk_text": (
        "The fund is exposed to equity market volatility, concentration risk in a limited "
        "number of stocks and sector risk. Investors may see significant interim declines; "
        "a minimum investment horizon of 5 years is recommended. Suitable for investors "
        "seeking long-term wealth creation who can tolerate market fluctuations."
    ),
    "fees": [
        ("Expense ratio - Regular plan", "1.62% p.a."),
        ("Expense ratio - Direct plan", "0.71% p.a."),
        ("Entry load", "Nil"),
        ("Exit load", "1% if redeemed within 365 days; Nil thereafter"),
        ("Stamp duty on purchase", "0.005%"),
    ],
    "performance": [
        ("1 year", "14.8%", "13.1%"),
        ("3 years (CAGR)", "16.2%", "15.0%"),
        ("5 years (CAGR)", "15.7%", "14.6%"),
        ("Since inception (CAGR)", "13.9%", "12.8%"),
    ],
    "eligibility": [
        ("Minimum lumpsum", "INR 5,000 and in multiples of INR 1"),
        ("Minimum additional purchase", "INR 1,000"),
        ("Minimum SIP", "INR 500 per month for at least 12 instalments"),
        ("SIP frequencies", "Daily, weekly, monthly, quarterly"),
        ("Who can invest", "Resident individuals, HUFs, companies, trusts, NRIs"),
        ("Lock-in", "None"),
    ],
    "taxation": [
        "Aurora is an equity-oriented scheme. Units held for 12 months or less are subject "
        "to short-term capital gains tax at 20%. Units held for more than 12 months are "
        "subject to long-term capital gains tax at 12.5% on gains above INR 1.25 lakh in a "
        "financial year.",
```

```
            "IDCW payouts are added to the investor's income and taxed at the applicable slab "
            "rate. TDS of 10% applies to IDCW above INR 5,000 for resident investors.",
        ],
        "faqs": [
            (
                "Is the Aurora Bluechip Equity Fund suitable for beginners?",
                "It can suit first-time equity investors with a horizon of 5 years or more, "
                "since it invests mainly in large, established companies. It still carries "
                "a Moderately High risk level.",
            ),
            (
                "Can I withdraw before one year?",
                "Yes. Redemptions within 365 days attract a 1% exit load and short-term "
                "capital gains tax of 20%.",
            ),
            (
                "Does the fund invest in small-cap stocks?",
                "Up to 20% of assets may be held in mid and small-cap equity.",
            ),
        ],
    },
    {
        "file": "meridian_short_duration_debt_fund.pdf",
        "name": "Meridian Short Duration Debt Fund",
        "category": "Debt - Short Duration Fund",
        "tagline": "An open-ended short-term debt scheme with Macaulay duration of 1 to 3
years",
        "facts": [
            ("Launch date", "22 August 2015"),
            ("Benchmark", "CRISIL Short Duration Debt A-II Index"),
            ("Fund manager", "Vikram Nair (since Jun 2021)"),
            ("AUM (30 Sep 2026)", "INR 3,150 crore"),
            ("NAV - Direct Growth", "INR 3,284.10"),
            ("Portfolio yield to maturity", "7.28%"),
        ],
        "objective": (
            "The scheme aims to generate regular income and moderate capital appreciation by
"
            "investing in a portfolio of debt and money market instruments such that the "
            "Macaulay duration of the portfolio is between 1 year and 3 years. There is no "
            "assurance that the objective will be achieved."
        ),
        "strategy": [
            "Meridian focuses on high-credit-quality accrual income. At least 85% of the "
            "portfolio is invested in AAA-rated or sovereign instruments, and the fund does "
            "not invest in instruments rated below AA+. Duration is actively managed within "
            "the mandated band based on the interest rate outlook.",
            "The average maturity of the portfolio is about 2.4 years and modified duration "
```

```
    "is about 1.9 years. The fund does not take leveraged positions or invest in "
    "unrated paper other than government securities.",
],
"allocation": [
    ("Government securities and T-bills", "20% - 50%", "Low"),
    ("AAA corporate bonds", "40% - 70%", "Low to Moderate"),
    ("AA+ and AA corporate bonds", "0% - 15%", "Moderate"),
    ("Money market and cash", "5% - 20%", "Low"),
],
"risk_level": 2,
"risk_text": (
    "The fund carries interest rate risk, as bond prices fall when yields rise, and "
    "a limited credit risk. It is not a guaranteed-return product. Suitable for "
    "investors seeking stable income with a horizon of 2 to 3 years who prefer lower "
    "volatility than equity funds."
),
"fees": [
    ("Expense ratio - Regular plan", "0.89% p.a."),
    ("Expense ratio - Direct plan", "0.31% p.a."),
    ("Entry load", "Nil"),
    ("Exit load", "0.25% if redeemed within 30 days; Nil thereafter"),
    ("Stamp duty on purchase", "0.005%"),
],
"performance": [
    ("1 year", "7.6%", "7.3%"),
    ("3 years (CAGR)", "7.4%", "7.1%"),
    ("5 years (CAGR)", "6.9%", "6.7%"),
    ("Since inception (CAGR)", "7.5%", "7.3%"),
],
"eligibility": [
    ("Minimum lumpsum", "INR 5,000 and in multiples of INR 1"),
    ("Minimum additional purchase", "INR 1,000"),
    ("Minimum SIP", "INR 1,000 per month for at least 6 instalments"),
    ("SIP frequencies", "Weekly, monthly, quarterly"),
    ("Who can invest", "Resident individuals, HUFs, companies, trusts, NRIs"),
    ("Lock-in", "None"),
],
"taxation": [
    "Meridian invests mainly in debt instruments and is taxed as a non-equity scheme. "
    "Capital gains, whether short-term or long-term, are added to the investor's "
    "income and taxed at the applicable income tax slab rate, regardless of the "
    "holding period.",
    "IDCW payouts are taxed at the slab rate. TDS of 10% applies to IDCW above "
    "INR 5,000 for resident investors.",
],
"faqs": [
    (
        "Are returns from the Meridian Short Duration Debt Fund guaranteed?",
```

```
            "No. Returns depend on interest rate movements and credit events, and the NAV "
            "can fluctuate even though volatility is lower than for equity funds.",
        ),
        (
            "What is the ideal holding period?",
            "2 to 3 years, aligned with the portfolio's Macaulay duration of 1 to 3 years.",
        ),
        (
            "Is there an exit load?",
            "A 0.25% exit load applies on redemptions within 30 days of purchase.",
        ),
    ],
},
{
    "file": "zenith_balanced_advantage_fund.pdf",
    "name": "Zenith Balanced Advantage Fund",
    "category": "Hybrid - Dynamic Asset Allocation Fund",
    "tagline": "An open-ended dynamic asset allocation scheme balancing equity and debt",
    "facts": [
        ("Launch date", "5 November 2018"),
        ("Benchmark", "CRISIL Hybrid 50+50 Moderate Index"),
        ("Fund manager", "Neha Iyer (equity), Vikram Nair (debt)"),
        ("AUM (30 Sep 2026)", "INR 5,760 crore"),
        ("NAV - Direct Growth", "INR 21.84"),
        ("Net equity exposure", "48% (as of 30 Sep 2026)"),
    ],
    "objective": (
        "The scheme seeks to provide capital appreciation and income by dynamically "
        "managing the allocation between equity and debt, based on a valuation model that "
        "tracks the price-to-earnings and price-to-book ratios of the market. There is no "
        "assurance that the objective will be achieved."
    ),
    "strategy": [
        "Zenith uses a proprietary valuation-based model. When market valuations are "
        "expensive, net equity exposure is reduced towards 30%; when valuations are "
        "attractive, it is raised towards 80%. The model is reviewed monthly by the "
        "investment committee.",
        "Gross equity exposure is held above 65% at all times by using arbitrage "
        "positions in the futures market to hedge the equity portion, so that the scheme "
        "qualifies for equity taxation. The debt portion is invested in high-quality "
        "government and AAA-rated securities.",
    ],
    "allocation": [
        ("Net unhedged equity", "30% - 80%", "Moderately High to High"),
        ("Arbitrage (hedged equity)", "10% - 40%", "Low to Moderate"),
        ("Debt and money market", "20% - 35%", "Low to Moderate"),
        ("Gold ETFs", "0% - 10%", "Moderate"),
    ],
```

```
        "risk_level": 3,
        "risk_text": (
            "The fund is exposed to equity market risk, interest rate risk and model risk, "
            "since the allocation depends on the valuation model's signals. The dynamic "
            "allocation is designed to dampen drawdowns but does not eliminate them. Suitable "
            "for investors seeking a smoother equity-linked experience over 3 to 5 years."
        ),
        "fees": [
            ("Expense ratio - Regular plan", "1.55% p.a."),
            ("Expense ratio - Direct plan", "0.66% p.a."),
            ("Entry load", "Nil"),
            ("Exit load", "0.5% if redeemed within 90 days; Nil thereafter"),
            ("Stamp duty on purchase", "0.005%"),
        ],
        "performance": [
            ("1 year", "11.2%", "10.4%"),
            ("3 years (CAGR)", "12.1%", "11.5%"),
            ("5 years (CAGR)", "11.6%", "11.0%"),
            ("Since inception (CAGR)", "11.9%", "11.3%"),
        ],
        "eligibility": [
            ("Minimum lumpsum", "INR 5,000 and in multiples of INR 1"),
            ("Minimum additional purchase", "INR 1,000"),
            ("Minimum SIP", "INR 500 per month for at least 12 instalments"),
            ("SIP frequencies", "Weekly, monthly, quarterly"),
            ("Who can invest", "Resident individuals, HUFs, companies, trusts, NRIs"),
            ("Lock-in", "None"),
        ],
        "taxation": [
            "Because Zenith maintains gross equity exposure above 65%, it is taxed as an "
            "equity-oriented scheme. Units held for 12 months or less are subject to "
            "short-term capital gains tax at 20%. Units held for more than 12 months are "
            "subject to long-term capital gains tax at 12.5% on gains above INR 1.25 lakh in "
            "a financial year.",
            "IDCW payouts are taxed at the applicable slab rate, with TDS of 10% above "
            "INR 5,000 for resident investors.",
        ],
        "faqs": [
            (
                "Why does the equity allocation of the Zenith Balanced Advantage Fund change?",
                "The fund raises or lowers net equity exposure based on market valuations, "
                "between 30% and 80%, while keeping gross equity above 65% through
arbitrage.",
            ),
            (
                "Is the fund taxed like an equity fund or a debt fund?",
                "Like an equity fund, because gross equity exposure stays above 65%.",
            ),
```

```
            (
                "Does the fund protect against market falls?",
                "It aims to reduce drawdowns, but capital is not protected and losses are
possible.",
            ),
        ],
    },
]

NAVY = colors.HexColor("#1F3A5F")
LIGHT = colors.HexColor("#EAF0F7")

_base = getSampleStyleSheet()
TITLE = ParagraphStyle("Title2", parent=_base["Title"], textColor=NAVY, fontSize=20,
spaceAfter=2)
SUBTITLE = ParagraphStyle(
    "Sub", parent=_base["Normal"], alignment=TA_CENTER, textColor=colors.grey,
fontSize=10
)
H2 = ParagraphStyle(
    "H2", parent=_base["Heading2"], textColor=NAVY, fontSize=13, spaceBefore=10,
spaceAfter=4
)
BODY = ParagraphStyle("Body2", parent=_base["BodyText"], fontSize=10, leading=14)
CELL = ParagraphStyle("Cell", parent=BODY, fontSize=9, leading=12)
CELL_BOLD = ParagraphStyle("CellBold", parent=CELL, fontName="Helvetica-Bold")
CELL_HEAD = ParagraphStyle(
    "CellHead", parent=CELL_BOLD, textColor=colors.white
)


def _table(rows, col_widths, header=None):
    data = []
    if header:
        data.append([Paragraph(h, CELL_HEAD) for h in header])
    for row in rows:
        data.append([Paragraph(str(c), CELL_BOLD if i == 0 else CELL) for i, c in
enumerate(row)])
    table = Table(data, colWidths=col_widths, repeatRows=1 if header else 0)
    style = [
        ("GRID", (0, 0), (-1, -1), 0.4, colors.lightgrey),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("ROWBACKGROUNDS", (0, 1 if header else 0), (-1, -1), [colors.white, LIGHT]),
        ("TOPPADDING", (0, 0), (-1, -1), 4),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
    ]
    if header:
        style.append(("BACKGROUND", (0, 0), (-1, 0), NAVY))
```

```
    table.setStyle(TableStyle(style))
    return table


def _riskometer(level_index):
    cells = [[Paragraph(name, CELL) for name in RISK_LEVELS]]
    table = Table(cells, colWidths=[17 * cm / len(RISK_LEVELS)] * len(RISK_LEVELS))
    palette = ["#7BC96F", "#B5D86B", "#F2E05B", "#F5B94F", "#F08A4B", "#D9534F"]
    style = [
        ("GRID", (0, 0), (-1, -1), 0.4, colors.white),
        ("ALIGN", (0, 0), (-1, -1), "CENTER"),
        ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
        ("TOPPADDING", (0, 0), (-1, -1), 8),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 8),
    ]
    for i, hex_color in enumerate(palette):
        style.append(("BACKGROUND", (i, 0), (i, 0), colors.HexColor(hex_color)))
    style.append(("BOX", (level_index, 0), (level_index, 0), 2.5, NAVY))
    table.setStyle(TableStyle(style))
    return table


def _footer(canvas, doc):
    canvas.saveState()
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(colors.grey)
    canvas.drawString(2 * cm, 1.2 * cm, DISCLAIMER)
    canvas.drawRightString(A4[0] - 2 * cm, 1.2 * cm, f"Page {doc.page}")
    canvas.restoreState()


def build_pdf(fund, path):
    doc = SimpleDocTemplate(
        str(path),
        pagesize=A4,
        leftMargin=2 * cm,
        rightMargin=2 * cm,
        topMargin=1.8 * cm,
        bottomMargin=2 * cm,
        title=fund["name"],
        author="Agentic RAG Experiments",
    )
    width = 17 * cm
    story = [
        Paragraph(fund["name"], TITLE),
        Paragraph(fund["category"], SUBTITLE),
        Spacer(1, 4),
        Paragraph(fund["tagline"], SUBTITLE),
```

```
        Spacer(1, 10),
        _table(fund["facts"], [width * 0.35, width * 0.65]),
        Paragraph("1. Overview and Investment Objective", H2),
        Paragraph(fund["objective"], BODY),
        Paragraph("2. Investment Strategy and Asset Allocation", H2),
    ]
    story += [Paragraph(p, BODY) for p in fund["strategy"]]
    story += [
        Spacer(1, 6),
        _table(
            fund["allocation"],
            [width * 0.45, width * 0.25, width * 0.30],
            header=["Instrument", "Allocation (% of net assets)", "Risk profile"],
        ),
        KeepTogether(
            [
                Paragraph("3. Risk Level (Riskometer)", H2),
                Paragraph(f"Risk level of the scheme: <b>{RISK_LEVELS[fund['risk_level']]}</b>",
BODY),
                Spacer(1, 4),
                _riskometer(fund["risk_level"]),
                Spacer(1, 6),
                Paragraph(fund["risk_text"], BODY),
            ]
        ),
        KeepTogether(
            [
                Paragraph("4. Fees and Charges", H2),
                _table(fund["fees"], [width * 0.4, width * 0.6]),
            ]
        ),
        KeepTogether(
            [
                Paragraph("5. Performance (as of 30 September 2026)", H2),
                _table(
                    fund["performance"],
                    [width * 0.40, width * 0.30, width * 0.30],
                    header=["Period", "Fund (Direct, Growth)", "Benchmark"],
                ),
                Paragraph("Past performance may or may not be sustained in the future.", CELL),
            ]
        ),
        KeepTogether(
            [
                Paragraph("6. Eligibility and Investment Minimums", H2),
                _table(fund["eligibility"], [width * 0.4, width * 0.6]),
            ]
        ),
```

```
        Paragraph("7. Taxation", H2),
    ]
    story += [Paragraph(p, BODY) for p in fund["taxation"]]
    story.append(Paragraph("8. Frequently Asked Questions", H2))
    for question, answer in fund["faqs"]:
        story.append(
            KeepTogether([Paragraph(f"<b>Q. {question}</b>", BODY), Paragraph(f"A.
{answer}", BODY)])
        )
        story.append(Spacer(1, 4))
    doc.build(story, onFirstPage=_footer, onLaterPages=_footer)


def main():
    DOCS_DIR.mkdir(exist_ok=True)
    for fund in FUNDS:
        path = DOCS_DIR / fund["file"]
        build_pdf(fund, path)
        print(f"Created {path}")


if __name__ == "__main__":
    main()
```

> in the terminal window,

> uv run scripts\generate_mf_docs.py

> now you should be able to see the docs folder having 3 MF PDF documents generated.

TRY LATER:

PROMPT used for the above task:

My name is Akash, part of the Core Development Team of Agentic RAG product.

I'm here to experiment how does RAG work with Agentic AI tools and planning and execution.

I want to create MF related product documents in docs folder, can you generate a python script that generates 3 MF product information as PDF documents. So I can use them as part of experiments.

Do you get my points? ask me questions so we both are clarified.

LangChain Documentation Related to How Agents are Built?

- Agent Development
- Agent Tools
- Agent Streaming
- Agent Retrieval for RAG

https://docs.langchain.com/oss/python/langchain/agents
https://docs.langchain.com/oss/python/langchain/tools
https://docs.langchain.com/oss/python/langchain/streaming
https://docs.langchain.com/oss/python/deepagents/retrieval

> in the vs code terminal window,

Execute the following command

> uv add "deepagents>=0.7" langchain langchain-openai langchain-chroma chromadb langchain-community langchain-text-splitters python-dotenv requests ipykernel

> create a folder named (at root) src\agentic_rag

> create a folder named notebooks at the root

> create a file named .env

> add following contents

OPENAI_API_KEY=your-openai-api-key
OPENAI_CHAT_MODEL=openai:gpt-5.5

Mock location returned by the get_location tool
LOCATION_CITY=Mumbai
LOCATION_COUNTRY=India
LOCATION_LATITUDE=19.0760
LOCATION_LONGITUDE=72.8777
LOCATION_TIMEZONE=Asia/Kolkata

Change the API key to

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

Create a file named rag.py and paste the following contents

```
"""Agentic RAG over the mutual fund PDFs: ingestion, tools, Deep Agent and streaming."""

import os
import re
from datetime import datetime
from pathlib import Path
from zoneinfo import ZoneInfo, ZoneInfoNotFoundError

import requests
from dotenv import load_dotenv
from langchain.messages import AIMessage, AIMessageChunk, ToolMessage
from langchain.tools import tool
from langchain_chroma import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain_core.documents import Document
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langgraph.checkpoint.memory import InMemorySaver
from pydantic import BaseModel, Field

from deepagents import (
    GeneralPurposeSubagentProfile,
    HarnessProfile,
    create_deep_agent,
    register_harness_profile,
)

PROJECT_ROOT = Path(__file__).resolve().parents[2]
DOCS_DIR = PROJECT_ROOT / "docs"
CHROMA_DIR = PROJECT_ROOT / "chroma_db"
COLLECTION_NAME = "mutual_funds"
EMBEDDING_MODEL = "text-embedding-3-small"
DEFAULT_CHAT_MODEL = "openai:gpt-5.5"

CHUNK_SIZE = 800
CHUNK_OVERLAP = 100
MAX_TOP_K = 6

HEADING = re.compile(r"^[1-8]\.\s+(.+?)\s*$", re.MULTILINE)
FOOTER = re.compile(r"Fictional product document.*?illustrative\.\s*(Page \d+)?",
re.DOTALL)

FILESYSTEM_TOOLS = frozenset({"ls", "read_file", "write_file", "edit_file", "delete", "glob",
"grep"})

WEATHER_CODES = {
```

```
    0: "clear sky", 1: "mainly clear", 2: "partly cloudy", 3: "overcast",
    45: "fog", 48: "rime fog", 51: "light drizzle", 53: "drizzle", 55: "heavy drizzle",
    61: "light rain", 63: "rain", 65: "heavy rain", 71: "light snow", 73: "snow",
    75: "heavy snow", 80: "rain showers", 81: "heavy rain showers", 82: "violent rain showers",
    95: "thunderstorm", 96: "thunderstorm with hail", 99: "severe thunderstorm with hail",
}

SYSTEM_PROMPT = """\
You are a warm, professional assistant for a catalogue of fictional mutual fund products.

SCOPE
You only help with four topics: (1) mutual fund product information from the knowledge base,
(2) weather, (3) location, (4) time.
For anything else (coding, news, general knowledge, other finance topics, opinions, etc.) do
NOT
answer. Reply with a humble, polite, warm and professional message saying you cannot
help with
that because this assistant is not designed for it, and invite the user to ask about mutual
funds, weather, location or time instead.

MUTUAL FUND QUESTIONS
- ALWAYS call `search_funds` before answering a fund question. Never answer from
memory.
- When the question names a fund or a topic (fees, risk, taxation, ...), use the `fund_name`
and
  `section` filters. For comparisons, call the tool once per fund.
- Cite fund name, section and page for every fact, e.g. (Aurora Bluechip Equity Fund, Fees
and Charges, p.1).
- If the retrieved context does not contain the answer, say so plainly. Do not guess.
- When you give a fund-related answer, remind the user that the funds are fictional and this
is not
  investment advice.

OTHER TOOLS
- `get_location` returns the user's current location (city, country, coordinates, timezone).
- `get_weather(city)` returns current weather and the city's IANA timezone.
- `get_time(timezone)` returns the current time for an IANA timezone such as Asia/Kolkata.
Chain tools when needed, e.g. weather and time for a city, then answer.
"""


def load_environment() -> None:
    load_dotenv(PROJECT_ROOT / ".env")
    if not os.getenv("OPENAI_API_KEY"):
        raise RuntimeError("OPENAI_API_KEY is not set. Copy .env.example to .env and add
your key.")
```

```
def get_vector_store() -> Chroma:
    return Chroma(
        collection_name=COLLECTION_NAME,
        embedding_function=OpenAIEmbeddings(model=EMBEDDING_MODEL),
        persist_directory=str(CHROMA_DIR),
    )


def _load_pdf_chunks(pdf_path: Path, splitter: RecursiveCharacterTextSplitter) ->
list[Document]:
    pages = PyPDFLoader(str(pdf_path)).load()
    first_lines = [ln.strip() for ln in FOOTER.sub("", pages[0].page_content).splitlines() if
ln.strip()]
    fund_name, category = first_lines[0], first_lines[1]

    chunks: list[Document] = []
    section = "Fund Facts"
    for page in pages:
        text = FOOTER.sub("", page.page_content)
        page_number = page.metadata["page"] + 1
        pieces: list[tuple[str, str]] = []
        position = 0
        for match in HEADING.finditer(text):
            pieces.append((section, text[position : match.start()]))
            section, position = match.group(1), match.start()
        pieces.append((section, text[position:]))

        for piece_section, piece_text in pieces:
            if not piece_text.strip():
                continue
            metadata = {
                "source": pdf_path.name,
                "page": page_number,
                "fund_name": fund_name,
                "category": category,
                "section": piece_section,
            }
            chunks.extend(splitter.create_documents([piece_text.strip()], metadatas=[metadata]))

    stem = pdf_path.stem
    for index, chunk in enumerate(chunks):
        chunk.id = f"{stem}-{index:03d}"
    return chunks


def ingest(store: Chroma | None = None) -> int:
    """Embed every PDF in docs/ into Chroma. Safe to re-run: ids are stable and stale chunks
are removed."""
```

```
    store = store or get_vector_store()
    splitter = RecursiveCharacterTextSplitter(chunk_size=CHUNK_SIZE,
chunk_overlap=CHUNK_OVERLAP)

    chunks: list[Document] = []
    for pdf_path in sorted(DOCS_DIR.glob("*.pdf")):
        chunks.extend(_load_pdf_chunks(pdf_path, splitter))
    if not chunks:
        raise FileNotFoundError(f"No PDFs found in {DOCS_DIR}")

    new_ids = [chunk.id for chunk in chunks]
    stale_ids = set(store.get()["ids"]) - set(new_ids)
    if stale_ids:
        store.delete(ids=list(stale_ids))
    store.add_documents(chunks, ids=new_ids)
    return len(chunks)


def _distinct(store: Chroma, key: str) -> list[str]:
    return sorted({m[key] for m in store.get()["metadatas"]})


def _match(value: str, options: list[str]) -> str | None:
    return next((o for o in options if o.lower() == value.strip().lower()), None)


class SearchFundsInput(BaseModel):
    query: str = Field(description="Natural-language search query")
    fund_name: str | None = Field(default=None, description="Restrict results to one fund")
    section: str | None = Field(default=None, description="Restrict results to one document
section")
    top_k: int = Field(default=4, description=f"Number of chunks to return (1-{MAX_TOP_K})")


def build_tools(store: Chroma) -> list:
    fund_names = _distinct(store, "fund_name")
    sections = _distinct(store, "section")

    search_description = (
        "Search the mutual fund product documents and return the most relevant passages
with "
        "fund, section and page for citation.\n"
        f"Valid fund_name values: {fund_names}.\n"
        f"Valid section values: {sections}."
    )

    def search_funds(query: str, fund_name: str | None = None, section: str | None = None,
top_k: int = 4) -> str:
```

```
        filters = []
        if fund_name:
            matched = _match(fund_name, fund_names)
            if not matched:
                return f"Unknown fund_name '{fund_name}'. Valid values: {fund_names}"
            filters.append({"fund_name": matched})
        if section:
            matched = _match(section, sections)
            if not matched:
                return f"Unknown section '{section}'. Valid values: {sections}"
            filters.append({"section": matched})
        where = None if not filters else filters[0] if len(filters) == 1 else {"$and": filters}

        top_k = max(1, min(top_k, MAX_TOP_K))
        docs = store.similarity_search(query, k=top_k, filter=where)
        if not docs:
            return "No matching passages found."
        return "\n\n".join(
            f"[{i}] Fund: {d.metadata['fund_name']} | Section: {d.metadata['section']} | "
            f"Page: {d.metadata['page']} | Source: {d.metadata['source']}\n{d.page_content}"
            for i, d in enumerate(docs, start=1)
        )

    search_tool = tool("search_funds", args_schema=SearchFundsInput,
description=search_description)(search_funds)

    @tool
    def get_location() -> str:
        """Get the user's current location: city, country, coordinates and IANA timezone."""
        return (
            f"City: {os.getenv('LOCATION_CITY', 'Mumbai')}, "
            f"Country: {os.getenv('LOCATION_COUNTRY', 'India')}, "
            f"Latitude: {os.getenv('LOCATION_LATITUDE', '19.0760')}, "
            f"Longitude: {os.getenv('LOCATION_LONGITUDE', '72.8777')}, "
            f"Timezone: {os.getenv('LOCATION_TIMEZONE', 'Asia/Kolkata')}"
        )

    @tool
    def get_weather(city: str) -> str:
        """Get the current weather for a city, plus the city's IANA timezone."""
        try:
            geo = requests.get(
                "https://geocoding-api.open-meteo.com/v1/search",
                params={"name": city, "count": 1},
                timeout=10,
            )
            geo.raise_for_status()
            results = geo.json().get("results")
```

```
            if not results:
                return f"Could not find a city named '{city}'."
            place = results[0]
            forecast = requests.get(
                "https://api.open-meteo.com/v1/forecast",
                params={
                    "latitude": place["latitude"],
                    "longitude": place["longitude"],
                    "current":
"temperature_2m,apparent_temperature,relative_humidity_2m,wind_speed_10m,weather_c
ode",
                },
                timeout=10,
            )
            forecast.raise_for_status()
            current = forecast.json()["current"]
        except requests.RequestException as error:
            return f"Weather service is unavailable right now: {error}"

        return (
            f"{place['name']}, {place.get('country', '')} (timezone {place.get('timezone',
'unknown')}): "
            f"{WEATHER_CODES.get(current['weather_code'], 'unknown conditions')}, "
            f"{current['temperature_2m']} C (feels like {current['apparent_temperature']} C), "
            f"humidity {current['relative_humidity_2m']}%, wind {current['wind_speed_10m']}
km/h"
        )

    @tool
    def get_time(timezone: str) -> str:
        """Get the current date and time for an IANA timezone, e.g. Asia/Kolkata or
Europe/London."""
        try:
            now = datetime.now(ZoneInfo(timezone))
        except (ZoneInfoNotFoundError, ValueError):
            return f"Unknown timezone '{timezone}'. Use an IANA name such as Asia/Kolkata."
        return now.strftime(f"%A, %d %B %Y, %H:%M:%S ({timezone}, UTC%z)")

    return [search_tool, get_location, get_weather, get_time]


def build_agent(store: Chroma | None = None):
    load_environment()
    store = store or get_vector_store()

    # Hide the default filesystem tools and the `task` tool; only our tools stay visible.
    register_harness_profile(
        "openai",
```

```
        HarnessProfile(
            excluded_tools=FILESYSTEM_TOOLS,
            general_purpose_subagent=GeneralPurposeSubagentProfile(enabled=False),
        ),
    )
    return create_deep_agent(
        model=os.getenv("OPENAI_CHAT_MODEL", DEFAULT_CHAT_MODEL),
        tools=build_tools(store),
        system_prompt=SYSTEM_PROMPT,
        checkpointer=InMemorySaver(),
    )


def stream_answer(agent, question: str, thread_id: str = "default") -> None:
    """Stream tokens inline, with labelled lines for tool calls and tool results."""
    config = {"configurable": {"thread_id": thread_id}}
    mid_line = False

    def break_line() -> None:
        nonlocal mid_line
        if mid_line:
            print()
            mid_line = False

    for chunk in agent.stream(
        {"messages": [{"role": "user", "content": question}]},
        config=config,
        stream_mode=["messages", "updates"],
        version="v2",
    ):
        if chunk["type"] == "messages":
            token, _ = chunk["data"]
            if isinstance(token, AIMessageChunk) and token.text:
                print(token.text, end="", flush=True)
                mid_line = True
        elif chunk["type"] == "updates":
            for node, update in chunk["data"].items():
                if node not in ("model", "tools") or not isinstance(update, dict):
                    continue
                for message in update.get("messages", []):
                    if isinstance(message, AIMessage) and message.tool_calls:
                        break_line()
                        for call in message.tool_calls:
                            print(f"\n[tool call] {call['name']}({call['args']})")
                    elif isinstance(message, ToolMessage):
                        break_line()
                        print(f"[tool result] {message.name}:\n{message.text}\n")
    break_line()
```

Create a file named agentic_rag.ipynb (Jupyter Notebook) in notebooks folder

> select the kernel

> select virtual environments

> select the workspace activated virtual environment

> if it prompts for firewall access permissions, click allow access

> click + code

```
from agentic_rag.rag import build_agent, build_tools, get_vector_store, ingest,
load_environment, stream_answer

load_environment()
```

> + code

```
store = get_vector_store()
print("chunks ingested:", ingest(store))
print("chunks in Chroma:", len(store.get()["ids"]))

sample = store.get(limit=3)
for metadata in sample["metadatas"]:
    print(metadata)
```

> + code

```
agent = build_agent(store)

# Filesystem tools and `task` are hidden from the model by the harness profile in rag.py.
print("tools exposed to the model:", [t.name for t in build_tools(store)])
```

> + code

```
# Single Document Lookup

stream_answer(agent, "What is the exit load of the Meridian Short Duration Debt Fund?",
thread_id="single")
```

> + code

```
# Multi-Document comparison

stream_answer(
    agent,
    "Compare the regular and direct plan expense ratios across all three funds in a table.",
    thread_id="compare",
)
```

> + code

```
stream_answer(agent, "How is the Zenith Balanced Advantage Fund taxed?",
thread_id="filtered")
print("\n" + "-" * 60 + "\n")
stream_answer(agent, "And what is its minimum SIP amount?", thread_id="filtered")
```

> + code

```
# How metadata filtering works

import re

search_funds = build_tools(store)[0]
query = "exit load"


def show(label, **filters):
    result = search_funds.invoke({"query": query, "top_k": 6, **filters})
    print(f"\n=== {label}  filters={filters or 'none'}")
    for header in re.findall(r"^\[\d+\] (.+)$", result, flags=re.MULTILINE):
        print("  ", header)


show("No filter (all funds, all sections)")
show("Fund filter", fund_name="Meridian Short Duration Debt Fund")
show("Fund + section filter", fund_name="Meridian Short Duration Debt Fund",
section="Fees and Charges")

print("\n=== Invalid filter value")
print(search_funds.invoke({"query": query, "fund_name": "Nonexistent Fund"}))
```

> + code

Multi-Tool Chain

```
stream_answer(
    agent,
    "What is the current time and weather in Mumbai? Based on that, which of the three funds would you "
    "describe as the lowest-risk option, and why?",
    thread_id="chain",
)
```

> + code

Location

```
stream_answer(agent, "Where am I, and what time is it here?", thread_id="location")
```

> + code

Refusals

```
stream_answer(agent, "Write me a Python function to reverse a linked list.",
thread_id="refuse-1")
print("\n" + "-" * 60 + "\n")
stream_answer(agent, "Who is the CEO of the Aurora Bluechip Equity Fund's asset management company?", thread_id="refuse-2")
```

TRY LATER:

Prompt used to generate RAG.py and Notebook Contents

Hey, can you analyze and understand the following URLs, so we shall be able to build agents with tools, streaming and importantly RAG capabilities.

https://docs.langchain.com/oss/python/langchain/agents
https://docs.langchain.com/oss/python/langchain/tools
https://docs.langchain.com/oss/python/langchain/streaming
https://docs.langchain.com/oss/python/deepagents/retrieval

NOTES:

- i would like to primarily use Chroma DB to keep my embeddings
- use OpenAI Embedding Models (text embedding 3 small)

- use .env for OpenAI API Key access

Do you understand my requirements? Do NOT generate any code immediately, and instead let's have a debate of MCQs based QAs so we both are clarified on what we want to build.

<ANSWER ALL QUESTIONS ASKED>

can you add one more question to the notebook next to the cell 6, to demonstrate how fund metadata specific filtering and search really works ...?

do you understand my requirement?